

Routing GitHub Copilot desktop app traffic through the local proxy — three approaches compared

2026-10-01. Three approaches came up this session for getting the Copilot desktop app's model traffic through `ai-gateway`'s local listener → Optimus, rather than straight to GitHub. This compares them and recommends one.

The underlying goal (stated explicitly mid-session): all model calls from the app route to the local proxy; the proxy knows which model was intended; the proxy has the real Copilot token; Optimus gets enough to let GitHub eventually serve the right model — with no need for a fictional custom provider in the app's own picker.

Approach 1: BYOK custom provider (direct SQLite write)

What it is. `src/subscribers/copilot_app_provision.rs` writes a `type='custom'` row into the app's own `~/.copilot/data.db` (`model_providers / provider_models`), pointing a `baseUrl` at the local listener. `listener::kinds::copilot_desktop` injects a real Copilot bearer (`listener::copilot_upstream_token`, now self-minting via the device flow — `copilot_upstream_token_mint.rs`) because the BYOK path carries no credential of its own.

Pros - The only approach that is **fully built and proven end-to-end against real Optimus** — `ai-testing/docs/copilot-desktop-real-optimus-e2e.md`, real `200`, real model output, 2026-09-29. - Pure userspace SQLite write. No kernel work, no system extension, no new listener transport. - Token handling is solved — auto-mint on top of the existing cache. - We control the model id end to end, so there's no ambiguity about what Optimus should map it to (today: one fixed id → `gpt-4o`).

Cons - **Only covers the custom provider's own traffic.** The real enterprise models (`claude-sonnet-5`, `gpt-6-sol`, etc., confirmed live via the `AIGuard0rg` Business seat) never touch it unless the user manually switches the picker to our entry — goal 1 ("all model calls") is not met. - **Forcing selection is a confirmed dead end.** `managed-settings.json`'s `model` key was live-tested twice: the device-MDM policy is parsed and logged (`keys=[model]`, `source=mdm`), but `app_state.copilot-selected-model` never changes. This reproduces the known `copilot-cli#4959` bug on the desktop app itself. The only mechanism actually confirmed to force selection is a **direct, manual** rewrite of `copilot-selected-model` plus each session's own `model / provider_id` columns — not automatic, not policy-driven. - **The model catalog is not durably editable on a real Business-seat account.** Deleting the real `github_copilot` provider row and trimming the available-models cache (the "X5" technique that worked pre-upgrade, 2026-09-24) got silently overwritten by a live account self-fetch on the very next launch — confirmed live, 2026-10-01. Local DB edits to the catalog do not survive this account's own sync behavior. - **Schema is reverse-engineered**, not GitHub-published — "nothing commits GitHub to this shape staying the same release to release" (`github-copilot-app-interception-feasibility.md`). The app has already auto-updated three times this session (`1.1.22` → `1.1.23` → `1.1.24` → `1.1.26`). - Requires quitting the app for every write (live WAL, concurrency risk). - Doesn't generalize past one hand-picked model id without more mapping work (the "X6" gap). - Explicitly contradicts the "no custom provider" goal — this approach *is* the custom provider.

Approach 2: Process-scoped HTTPS_PROXY

What it is. Launch the app with `HTTPS_PROXY / HTTP_PROXY` set for that one process. Its own HTTP client (confirmed request-based) tunnels *all* its outbound HTTPS through the local listener via `CONNECT`; the listener terminates TLS with a locally-trusted CA and relays, seeing the real request — real model name in the body, real bearer the app already carries in `Authorization`.

Pros - Confirmed working, live, this session. Launching the real binary with `HTTPS_PROXY` set produced a captured `CONNECT api.github.com:443` from the genuine app (`user-agent: github-app/1.1.26`) — proof the app's client honors standard proxy env vars. - **Covers every model, with zero mapping problem.** The real model id (`claude-sonnet-5` , etc.) is already in the request body the listener reads — no fictional provider, no id translation, no picker-forcing fight. - **No SQLite writes at all** — none of approach 1's schema-fragility, concurrency, or live-resync problems apply. - **The token problem mostly disappears.** Unlike the BYOK path, the app's first-party calls already carry a real bearer; the listener can forward it unchanged, the same pattern every *other* brokered agent already uses (`http / https` listener kinds) — `copilot_desktop` 's auth-injection exists only because BYOK has no credential of its own. - Reuses an existing pattern in this codebase (`LlmRouting::ForwardProxy` , already shipping for Copilot CLI/Gemini CLI/etc.) — architecturally consistent, not a one-off.

Cons - No durable "every launch" story on macOS yet. The desktop app has no persisted proxy setting of its own to write once (unlike the CLI tools that read a `proxy / proxyUrl` config key every start). `launchctl setenv` is session-wide; editing `Info.plist / LSEnvironment` or swapping the bundle's executable both break code signing and get wiped by the app's own auto-updates. The only thing that works today is a wrapper script the user has to launch *instead of* the real icon — a real behavior change, not true enforcement. - Requires the app to trust a locally-generated CA for TLS termination — a real, ongoing security-surface decision (whoever holds that CA's key can decrypt anything the app trusts it for). - Still depends on the app's own cooperation (honoring the env var) — lower risk than approach 1's reverse-engineered schema (respecting proxy env vars is a basic, widely-relied-on client behavior), but not contractually guaranteed by GitHub either. This is the one limitation `network_egress` 's redirect work (later phase) actually removes — see the Recommendation.

Update, 2026-10-05 — both remaining open items above are now closed, confirmed live, not just designed:

- **The TLS-terminating relay is built**, and needed no new listener kind at all — it reuses the existing, already-shipping `listener::kinds::https` forward-proxy machinery (the same one Claude Code's `HTTPS_PROXY` leg uses: generic `CONNECT` handling, per-host TLS termination via an on-the-fly-minted leaf, no allowlist, no auth injection). The new code is a small standalone harness, `ai-gateway/copilot-desktop-forward-proxy` , that binds it and signs with a borrowed already-enrolled agent's real triple JWT — modeled directly on `copilot-desktop-simple` . - **A real inference call through real Optimus is confirmed**, live: with the app launched under this harness's `HTTPS_PROXY` , a prompt sent to a real enterprise model produced `POST /v1/messages -> api.business.githubcopilot.com http/1.1 200` — real triple-JWT signing, real backend, real response. (The wire shape turned out to be `/v1/messages` — Anthropic's Messages API format — for every model family, not `/chat/completions / /responses` as earlier assumed; GitHub's backend normalizes to one shape regardless of the underlying provider.) Every other host the app needs — `api.github.com` , `exp.business.githubcopilot.com` , `telemetry.business.githubcopilot.com` , even unrelated ones like `ai.azure.com / api.catalog.azureml.ms` — relayed through the same listener with no special-casing required, confirming the "no allowlist" design claim empirically rather than just architecturally. - One operational gotcha hit during this test, worth recording: an existing chat session can carry its own stale `provider_id / model` reference from earlier BYOK testing (the `sessions` table's own stickiness, not a new bug) — pointing it at a port nothing

listens on anymore produces a `Connection reset by peer` error that looks like a new failure but isn't. Fixed by either reselecting a real model in that session or starting a fresh one (new sessions pick up the global default with no manual step).

Three more operational gotchas hit immediately after the above, worth recording precisely since they cost real debugging time before being isolated — none are architectural problems, all are harness/environment friction:

1. **The harness minted a brand-new CA on every restart**, silently invalidating whatever the operator had just trusted in Keychain. Fixed in code: it now persists `cert_pem / key_pem` to disk on first run and reuses them on every subsequent run, matching the real product's own `local_ca_credentials` doc comment ("created on first use and reused thereafter"). Restarting the harness no longer requires re-trusting anything.
2. **macOS Keychain trust got ambiguous across two CAs sharing an identical subject DN** (`CN=Zscaler Agentic Local Root CA` — the fixed subject `gateway_util::ca::generate()` always mints). `security find-certificate / dump-trust-settings` returned confusing, seemingly contradictory results while two certs with that same subject but different keys coexisted. Resolved by deleting the stale one by its exact SHA-256 fingerprint before re-adding the current one — collisions on subject text, not the actual key, are the thing to watch for here. `security add-trusted-cert -r trustRoot -k <keychain> <ca.pem>` against the login keychain works without `sudo` and is more reliable than the Keychain Access GUI import flow for this.
3. **A real triple JWT got rejected within under a minute of minting it**, far faster than the ~20-minute window observed earlier — turned out to have nothing to do with the credential at all. A `curl` made *directly through the harness* (`curl -x http://127.0.0.1:<port> --cacert <ca.pem> https://...`) surfaced the real response body, which the harness's own logs don't capture (status/timing only): a genuine Zscaler block page (`server: Zscaler/6.2`), not an Optimus 403 body at all. **A second, separate Zscaler client — /Applications/Zscaler/ Zscaler.app plus its com.zscaler.zscaler.TRPTunnel system extension — was intercepting this machine's traffic to gateway.zsagenti.c.a.i**. This is distinct from "ZCC" (the client this session has otherwise turned off repeatedly for Optimus testing) — confirmed by process start time (`ps -o lstart`) lining up almost exactly with the gap between a successful run and the first failure, and by the fact that disabling ZCC alone left this one still running. Stopping it (no menu-bar "disconnect" option existed; required killing the processes directly, which **is a real MDM-policy question, not a casual workaround** — this profile sets `OnDemandUserOverrideDisabled: 1`, so treat doing this as something to clear with whoever owns that policy before relying on it again, not a standing procedure) resolved it immediately, confirmed by a clean `curl` response before even relaunching the app.

The debugging technique worth keeping, independent of this specific incident: when something fails with an opaque status code and the harness's own log only has status + timing, `curl through the harness itself` (as a real CONNECT-proxy client, using the harness's own trusted CA) gets the actual response body in seconds — that's what separated "a Zscaler block page" from "an Optimus 403" here, and nothing short of capturing the real body would have told the two apart.

Approach 3: `network_egress` (per-process allow/block)

What it is. `ai-protect`'s existing settings-controlled feature: kernel-level rules matching **process + destination domain/IP**, with `allow > block > audit` precedence (`docs/network-egress.md`). A real policy — allow the

Copilot app to reach `api.githubcopilot.com`, block every other process from the same destination — needs no new code on Windows.

Important distinction from 1 and 2: this doesn't route anything. It can only `allow` (do nothing) or `block` (hard-deny) a connection. It cannot get the app's traffic to localhost — it's a containment/hardening control, not an alternative transport. It answers a different question ("can an unauthorized process reach a model host directly") than approaches 1 and 2 answer ("how does the *authorized* app's traffic reach Optimus").

Pros - Already shipping, on Windows — zero new engineering, pure rule authoring (`processes` + `domains` dimensions, confirmed real, enforced via `FWPM_CONDITION_ALE_APP_ID`). - Kernel-enforced, not app-cooperation-dependent — a non-whitelisted process genuinely cannot complete the connection, unlike approaches 1/2 which both rely on the Copilot app choosing to use the mechanism we've set up. - Mature, production-grade subsystem (ETW + WFP), not a reverse-engineered private schema.

Cons - macOS enforcement doesn't exist. Confirmed by direct investigation: daemon-side plumbing exists (`src/services/net_egress/macos.rs`, shared policy evaluator) and the hardest Apple hurdle — the NE entitlement + provisioning profile — looks already cleared, but the actual `NEFilterDataProvider` extension, its signing/build tooling, and any *block* semantics (the existing macOS design is observe-only) are all unbuilt. Real, multi-piece systems work, not a quick add. - **Doesn't address goals 2–4 at all.** Even fully built, it says nothing about model-awareness, token handling, or what Optimus receives — those are entirely properties of whichever of approach 1 or 2 is actually carrying the traffic. - Only meaningful paired with 1 or 2 — on its own it can make the Copilot app's direct path *fail*, not succeed through the proxy.

Recommendation — adopted plan (2026-10-02, ratified by the team 2026-10-04)

Update, 2026-10-04 — this is now an actual team decision, not just this document's own engineering recommendation. Confirmed at a team meeting: the plan is to test a solution based on `HTTPS_PROXY` for the short term, with the team explicitly accepting that it is **not enforceable** and **depends on end-user cooperation** — the same limitation this document already names (no durable "every launch" story on macOS, a human can bypass it by launching the real icon). For the short term, that's accepted as an okay tradeoff, not an oversight. For the long term, the plan is to **socialize** a `network_egress` -hook-based solution — deliberately not "build" yet. That word matters: the long-term piece still needs buy-in from whoever else it touches before it's a committed engineering task, distinct from `HTTPS_PROXY`, which is already approved to actually test.

Decision: approach 2 (`HTTPS_PROXY`) now, approach 3 (`network_egress` redirect) later. Approach 1 (BYOK) is dropped — not just de-prioritized, not even kept as a fallback. Earlier this document hedged with "use approach 1 as the near-term fallback" in case approach 2's listener wasn't ready in time for the Windows VM test. That hedge is withdrawn.

The canonical reason BYOK is closed, stated precisely: it requires **undocumented, reverse-engineered SQLite modifications** to the Copilot desktop app's own `data.db` — a schema GitHub never published and never committed to keeping stable release to release — and **the app's own enterprise-subscription sync can silently overwrite those changes** (confirmed live: a real Business-seat account's model-catalog resync restored every deleted row on the very next launch, undoing the edit entirely). Those two properties together — unsupported modification of a private schema, plus a live, out-of-our-control sync that can erase it at any time — are why this is

dropped outright, not deprioritized. The unforced-picker-selection problem and the confusing duplicate provider entry in the UI, both observed during testing, are downstream symptoms of this, not separate root causes — and none of them are fixable without approach 2's own machinery anyway, so there's no scenario where shipping approach 1 first would have saved real time.

Why this is better than the comparison above initially suggested, not just equally good: skipping the database entirely removes every problem that actually consumed this session's debugging time — there is no catalog to keep durable against the live resync, no picker-forcing fight, no schema to keep re-validating against the app's auto-updates, and no risk of a stale `provider_id` reference showing "(not available)" in an old session. It also looks cleaner in the UI: users see their real models, under their real names, with no parallel "GPT-4o via Zscaler AI Protect" entry sitting alongside them — the brokering is invisible rather than a visible, separate choice someone has to make correctly every time.

What's already de-risked, not just hoped for: the app's HTTP client genuinely honors a process-scoped `HTTPS_PROXY` for its real first-party traffic (captured live, including the actual `/models` call and its real response). The app's first-party requests already carry their own valid bearer, so the listener has nothing to mint — pass-through only, same as every other brokered agent. And the CA-trust question is answered, not assumed: the live `/models` capture succeeded with a real `200` through a CA installed into the system keychain, meaning the app's Rust backend does accept a keychain-trusted CA for TLS interception — exactly the mechanism a real listener needs.

What's genuinely new work, with one nuance worth not glossing over: a new listener kind terminating TLS for `api.business.githubcopilot.com`, routed by Host (the client is talking to the real hostname now, not a `baseUrl` override), passing the client's own `Authorization` through, signing with the triple JWT, relaying through Optimus. This is closer in shape to the existing `Https / Dual` listener transport (already used for other agents' forward-proxy legs) than to the BYOK-specific `copilot_desktop` kind, and should be built as a sibling to that infrastructure. The nuance: `agent-manager`'s existing `LlmRouting::ForwardProxy` (what Copilot CLI/Gemini CLI use) looked reusable at first, but only half of it transfers — it writes the proxy URL into the *agent's own persisted config file*, which is what makes it durable on every launch for those tools. The desktop app has no such field (checked `config.json` and every `app_state` key — nothing proxy-related exists), so only `ForwardProxy`'s listener-side half (the TLS/local-CA infra) applies here; the file-writing half does not.

"Now" vs. "later," stated precisely, matching the phasing actually adopted: *now* means a manually-set `HTTPS_PROXY` env var, same mechanism validated live this session — no durable "every launch" story yet, and that gap is known and accepted for this phase, not overlooked. *Later* means `network_egress`'s redirect extension (the `NETtransparentProxyProvider` / WFP-callout design docs) removes that dependency entirely, enforcing the same routing regardless of launch method. Until then, on Windows, plain `network_egress` block/allow (already shipping, zero new code) is worth authoring as a **hardening layer now**, even before the redirect work lands — it doesn't replace the `HTTPS_PROXY` routing, but it closes the "a user just bypasses it by launching normally" gap on one platform today, which the redirect work will later close everywhere.

Concretely, next steps, updated 2026-10-05 — (1) and (2) are done: (1) `~~build the new listener kind~~` — done, and turned out to need no new listener kind at all: `listener::kinds::https` already did everything required, reused as-is by the new `copilot-desktop-forward-proxy` harness; (2) `~~prove one real enterprise model end-to-end through real Optimus~~` — done, confirmed live (`POST /v1/messages -> api.business.githubcopilot.com http/1.1 200`), not just direct curl. Remaining: (3) author the Windows `network_egress` block/allow rule as the near-term hardening layer, since it costs nothing new; (4) decide whether `copilot-desktop-forward-proxy`'s approach gets wired into `agent-manager` as a real, shippable `LlmRouting`

variant (a bigger change — no existing variant quite fits, since neither `BaseUrl` , `ForwardProxy` 's file-writing half, nor `CopilotDesktopAuthRelay` apply) or stays a dev harness until the `network_egress` redirect work removes the "manually-launched" dependency; (5) pursue the `network_egress` redirect work itself (both design docs) as the durable "every launch" fix — per the team's 2026-10-04 decision, this is to be *socialized*, not yet a committed build.

Appendix: detailed Q&A (2026-10-02)

Fourteen specific questions, answered in order, each grounded in what was actually confirmed this session (or in the two sibling design docs, `copilot-desktop-nettransparentproxyprovider-design.md` and `copilot-desktop-wfp-redirect-callout-design.md` , linked rather than repeated where the answer is platform-mechanics detail).

1. What is the schema of the SQLite tables?

Three tables, confirmed verbatim from `src/subscribers/ copilot_app_provision.rs` 's module doc and cross-checked against a live, read-only query of a real `~/ .copilot/data.db` :

```
CREATE TABLE model_providers (
  id TEXT PRIMARY KEY NOT NULL,
  name TEXT NOT NULL,
  created_at TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ', 'now')),
  updated_at TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ', 'now')),
  type TEXT NOT NULL DEFAULT 'openai',
  settings_json TEXT NOT NULL DEFAULT '{}',
  account_id TEXT REFERENCES accounts(id) ON DELETE CASCADE
);
CREATE TABLE provider_models (
  id TEXT PRIMARY KEY NOT NULL,
  provider_id TEXT NOT NULL REFERENCES model_providers(id) ON DELETE CASCADE,
  model_id TEXT NOT NULL,
  wire_model TEXT,
  display_name TEXT NOT NULL,
  max_prompt_tokens INTEGER,
  max_output_tokens INTEGER,
  wire_api_override TEXT CHECK (wire_api_override IS NULL OR wire_api_override IN ('completions', '
  created_at TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ', 'now')),
  updated_at TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ', 'now')),
  supported_reasoning_efforts TEXT,
  UNIQUE (provider_id, model_id)
);
CREATE TABLE app_state (
  key TEXT PRIMARY KEY NOT NULL,
  value TEXT NOT NULL,
  updated_at TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ', 'now'))
);
```

A fourth table, `sessions` , matters too (it's what makes an *existing* chat sticky to a model/provider independent of the global default) — but **its schema is not documented anywhere in this codebase**, unlike the three above. The only in-repo trace is a leftover permission entry in `.claude/settings.local.json` from an earlier ad-hoc

sqlite3 query against the live app DB — i.e. tribal knowledge, not a maintained doc comment. Read directly off a live install for this doc:

```
CREATE TABLE sessions (
  id TEXT PRIMARY KEY NOT NULL, title TEXT,
  session_type TEXT NOT NULL DEFAULT 'workspace', mode TEXT, model TEXT,
  reasoning_effort TEXT, is_running INTEGER NOT NULL DEFAULT 0,
  was_interrupted INTEGER NOT NULL DEFAULT 0, interruption_reason TEXT,
  created_at TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ','now')),
  updated_at TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ','now')),
  remote_control_enabled INTEGER NOT NULL DEFAULT 0, auto_approve INTEGER NOT NULL DEFAULT 1,
  enabled_experiments_json TEXT, forked_from_session_id TEXT,
  fork_original_history_event_count INTEGER,
  total_input_tokens INTEGER NOT NULL DEFAULT 0, total_output_tokens INTEGER NOT NULL DEFAULT 0,
  total_cached_tokens INTEGER NOT NULL DEFAULT 0, total_reasoning_tokens INTEGER NOT NULL DEFAULT 0,
  total_nano_aiu INTEGER NOT NULL DEFAULT 0, context_current_tokens INTEGER,
  context_input_token_limit INTEGER, context_output_token_limit INTEGER,
  remote_session_mode TEXT, agent TEXT,
  provider_id TEXT NULL REFERENCES "model_providers"(id) ON DELETE SET NULL,
  remote_connect_target TEXT, execution_location TEXT NOT NULL DEFAULT 'local',
  context_tier TEXT, context_system_tokens INTEGER, context_conversation_tokens INTEGER,
  context_tool_definitions_tokens INTEGER, context_mcp_tools_tokens INTEGER, context_buffer_tokens INTEGER,
  title_source TEXT NOT NULL DEFAULT 'auto' CHECK (title_source IN ('auto','agent','user')),
  total_agent_merge_nano_aiu INTEGER NOT NULL DEFAULT 0, archived_at TEXT,
  permission_mode TEXT CHECK (permission_mode IN ('off','auto','on')),
  remote_availability TEXT, remote_availability_revision INTEGER NOT NULL DEFAULT 0,
  remote_create_state TEXT, remote_prompt_delivery TEXT, sandbox_enabled INTEGER,
  recovered_empty_execution_event_id TEXT, provisional INTEGER NOT NULL DEFAULT 0,
  empty_execution_failure_event_json TEXT, first_turn_context_delivery TEXT,
  lifecycle_generation INTEGER NOT NULL DEFAULT 0, agent_scope TEXT,
  remote_lifecycle_owned INTEGER NOT NULL DEFAULT 0
);
```

If this table's shape is going to be relied on going forward, it deserves the same doc-comment treatment `copilot_app_provision.rs` already gives the other three, rather than staying tribal knowledge.

2. What changes are made to the data?

Shipped, real product code (`copilot_app_provision.rs` , on agent register/deregister): - `INSERT` into `model_providers`: `id = "custom:zscaler-ai-protect"`, `type = "custom"`, `settings_json = {"authKind":"none","baseUrl":"http:// 127.0.0.1:<port>","headersJson": {"},"wireApi":{"completions"}}`. - `INSERT` into `provider_models`: `provider_id = "custom:zscaler-ai-protect"`, `model_id = "gpt-4o"` (a hardcoded constant — see Q7), `display_name = "GPT-4o via Zscaler AI Protect"`. - `UPDATE` of **both** `app_state` keys named in `AVAILABLE_MODELS_KEYS` (`copilot-cloud-available-models-v2` and `copilot-available-models-v2`), appending `{"capabilities":{"supports":{"reasoningEffort":false}}}`, `"id":"custom:zscaler-ai-protect/gpt-4o"`, `"name":"GPT-4o via Zscaler AI Protect"`, `"providerId":"custom:zscaler-ai-protect"}`.

Ad hoc, this session only, never shipped as product code: - Deleting every other

`model_providers` / `provider_models` row and trimming both `app_state` keys to just our entry — proven **not durable**: a live account self-fetch silently restored the full 28-model catalog on the very next launch. - Directly rewriting `app_state.copilot-selected-model` plus an existing session's own `model` / `provider_id` columns —

the historical "X5" test (2026-09-24) confirmed this is the **one mechanism that actually forces selection**, for both fresh and existing sessions, but it's a manual, one-time write, not something that happens automatically.

3. What that solves, and what it doesn't

Solves: a real, selectable model entry that is fully proven end-to-end through real Optimus (`ai-testing/docs/copilot-desktop-real-optimus-e2e.md` — genuine 200 , genuine model output).

Does not solve: - Forcing the app onto it. `managed-settings.json` 's `model` key is a confirmed dead end (parsed and logged, never applied to selection — reproduces `copilot-cli#4959` on the desktop app itself). - Covering the real enterprise models' own traffic at all — they bypass this entirely unless a human manually switches the picker. - Surviving catalog-level DB edits — the live resync undoes them. - Schema stability — reverse-engineered from the shipped binary, not GitHub-published; the app has auto-updated three times this session alone.

4. Why is `HTTPS_PROXY` required?

Because nothing else reaches the traffic it reaches. The real first-party provider's `baseUrl` is empty and, as far as anything tested this session shows, not read for a `type='github_copilot'` row the way it is for `type='custom'` — there is no local config field confirmed to redirect it. Direct DB edits to the catalog don't survive the account's live resync either way. The **one** confirmed lever that reaches this traffic is the environment variable the app's own Rust backend already honors (confirmed live: a process-scoped `HTTPS_PROXY` produced a real, captured `CONNECT` from the genuine app, and later a real, captured `GET /models` with its actual response body) — it's required because it's the only thing that actually works, not because it's the most elegant option.

5–6. How is `network_egress` better than `HTTPS_PROXY` , and how is it different?

Better: `HTTPS_PROXY` only takes effect because we controlled how the process launched. `network_egress` (specifically its redirect-extended form — see Q14) is enforced at the kernel/network-extension layer, independent of launch method — double-clicking the real Dock icon doesn't bypass it the way it bypasses an env var we didn't get to set.

Different, and this is the part worth not glossing over: they aren't actually the same capability yet. The `network_egress` feature that is **already shipping today** (Windows only) can only `allow` or `block` a connection — it cannot redirect it to localhost at all. So "better than `HTTPS_PROXY` " is only true once it's paired with the new redirect work (`NETTransparentProxyProvider` / the WFP callout, both still design docs, not code). Plain, already-shipped `network_egress` solves a narrower problem — guaranteeing no bypass — not the same problem `HTTPS_PROXY` solves today (actually rerouting traffic and serving a crafted response).

7. How many models need to be added to the UI? One per model in `/models` ?

Today: **exactly one** — `MODEL_ID = "gpt-4o"` is a hardcoded constant in `copilot_app_provision.rs` , not derived from the real catalog at all.

To mirror the *full* real catalog under our own provider (so a user could pick "Claude Sonnet 5" and have it genuinely route through us under its real name), the schema requires **one provider_models row per model** — `UNIQUE (provider_id, model_id)` means each selectable entry is its own row. So yes, in principle one-for-one with

whatever the real `GET https://api.business.githubcopilot.com/models` response contains (28 entries, confirmed live) — this is unbuilt; today there is only the one.

8. Is Copilot token acquisition done in the localhost LLM proxy?

Only for the BYOK path. `listener::copilot_upstream_token_mint` runs *inside* the listener process itself, performing the GitHub OAuth device flow and writing the result to `~/.zsai-gateway/copilot-upstream-token.txt` — because BYOK carries no credential of its own, something has to supply one, and that's what this does.

Not needed at all for first-party interception. The real app's own first-party requests already carry a valid credential — confirmed live via the mitm capture: `authorization: Bearer gho_...` on the real `GET /models` call, supplied by the app itself. A listener intercepting that traffic has nothing to mint; its job is to pass the client's own header through unchanged, the same pattern every other brokered agent's listener already uses.

9. Everything done in "localhost" — including the SDK steps

For the one architecture that's fully built (BYOK), in order:

1. **One-time identity bootstrap** (`zax_sdk::enroll`, run via `activate-agent` or the real `agent-manager`): register with Vector, poll for admin approval, activate with Bumblebee (mints `instance_jwt`), token-exchange with Bumblebee (mints the triple JWT) — persisted to `~/.zsai-gateway/.credentials.zip` via `creds-manager`. Separate from, and prior to, any per-request work.
2. **Upstream-token acquisition** (a *different* credential from the triple JWT): the GitHub OAuth device flow → a real Copilot bearer → `~/.zsai-gateway/copilot-upstream-token.txt`.
3. **Per request, at the listener:** receive the Copilot app's request on the bound local port → `inject_auth` overwrites `Authorization` with the real Copilot bearer and stamps `editor-version / editor-plugin-version / copilot-integration-id` → `forward() / Gateway::send` signs with the triple JWT (RFC 9421 request signing, `X-ZAX-Authorization`) and relays over the gateway's own TLS connection to Optimus → Optimus validates and forwards upstream to the real `api.githubcopilot.com` → response relayed back unmodified.

10. Everything done in `ai-protect`

- `src/subscribers/copilot_app_provision.rs` — the BYOK SQLite provisioning (register/deregister the custom provider, update both `app_state` keys).
- `src/utils/sqlite_provision.rs` — the generic, foreign-database safety primitive it's built on.
- `src/utils/atomic_write/` — the atomic file-write primitive it reuses.
- `src/subscribers/ai_broker_launch.rs` — special-cases `copilot-desktop`'s config-delta dispatch so it reaches `copilot_app_provision` instead of the generic config-file-merge path every other agent uses.
- `src/utils/discovery/schema.rs` — the `ClientKind` entry that makes the desktop app discoverable at all, distinct from the `copilot` CLI entry (fixed separately, during the real VM test — see `TOKEN_EXCHANGE_AUTH_BUG.md`).
- `src/services/net_egress/macos.rs` — daemon-side plumbing for the **not-yet-built** macOS `network_egress` enforcer (lifecycle, app-group policy file, outbox tailer) — real code, but with no actual `NEFilterDataProvider` extension behind it yet.

11. Everything done in ai-gateway

- `agent-manager` : `AgentSpec / LlmRouting::CopilotDesktopAuthRelay / ListenerTransport::CopilotDesktopAuthRelay` (`agents/copilot_desktop.rs` , `agents/mod.rs` , `agent.rs`), `ENABLED_AGENTS` inclusion, and `enrol.rs::registry_agent_type` 's `copilot-desktop` → "custom" mapping for Vector registration.
- `listener` : `kinds/copilot_desktop.rs` (auth-injecting listener kind), `copilot_upstream_token.rs` + `copilot_upstream_token_mint.rs` (token cache and self-minting), reusing `gateway.rs` 's existing TLS client/connection pooling to Optimus.
- `sdk` (`zax_sdk`): the enroll pipeline, including this session's real fix to `token_exchange()` 's missing `Authorization` header (`41aa2bc8`).
- `creds-manager` : persists the resulting triple-JWT/HMAC credentials.
- Dev-harness binaries: `activate-agent` , `copilot-desktop-simple` , `simulate-ai-gateway` , and the `simulator` crate (binary `zax-sim` , explicitly marked "Research VM only" — plays *ai-protect* 's side of the file protocol, reacting to `.status.zip` /config changes and rewriting `.broker.zip`). `simulate-ai-protect` is closely related but actually lives in the `ai-protect` root, not under `ai-gateway/` .
- `ai-gateway/scripts/get_copilot_token.sh` exists, but on the **sibling** `main-08272026-vscode-copilot` branch — not merged into this branch.

12. ai-gateway 's launch modes

The real binary (`zscaler-ai-gateway` , crate `ai-gateway-service` , `ai-gateway/service/src/main.rs`) parses its own args — not `clap` — into:

Invocation	Mode	What it does
<i>(no args)</i> , or <code>--service</code>	Service	Continuous: watches <code>.broker.zip</code> , enrolls each agent, serves it, writes <code>.status.zip</code> . What <code>ai-protect</code> 's daemon actually spawns.
<code>--agents</code>	Agents	One-shot: prints <code>agent_manager::agents::diagnose()</code> (id/enabled/installed/mcp/llm) and exits. No config/trust built.
<code>--recover</code>	Recover	One-shot: undoes this product's config edits. Takes <code>--dry-run</code> , <code>--purge-home</code> , <code>--clean-backups</code> , <code>--keep-credentials</code> .
<code>--sim-daemon</code>	SimDaemon	One-shot dev harness: plays <code>ai-protect</code> 's file-protocol slice.
<code>--help / -h</code>	—	Prints help, exits.

Modes are mutually exclusive. Notably, several older flags are **explicitly rejected** now, not silently reinterpreted: `--mcp-server` , `--llm-proxy` , `--llm-forward-proxy` , `--configure-llm` , `--base-url` , `--control` all fail as "unknown argument" — a deliberately trimmed surface, worth knowing if an old doc or habit still references one of them.

Excluded per Q13: `--bridge` AGENT (the `Bridge` mode — a continuous MCP stdio relay for one agent session) exists and is real, but is out of scope for this document by request.

13. (Scope note, not a question) — MCP excluded

Noted above and applied throughout: `--bridge` /MCP relay behavior is deliberately not covered in this document's analysis.

14. The role of `network_egress` — Windows vs. macOS in full

The role: the containment layer ensuring model-provider traffic can only reach its destination via the sanctioned path — either by denying direct bypass outright (what's shipped today) or by actively rerouting it (the new, unbuilt redirect work in the two sibling design docs).

"Kernel-level" means something different on each platform, and it's not just a detail: - **Windows has two tiers under one umbrella.** Today's `block / allow` is kernel-enforced but needs **no custom kernel code at all** — WFP is a rich, OS-native, declarative rule engine; third parties register conditions and get Microsoft's own built-in permit/block actions applied by the kernel's existing, already-signed WFP engine. Only the *new* redirect capability requires shipping genuinely custom kernel-mode code (a callout, `FwpsCalloutRegister1` +) — that's the first kernel driver this feature would ever need. - **macOS has no such declarative tier at all.** There is nothing on macOS analogous to "register a condition, get a stock kernel action" — Apple removed third-party kernel extensions for this class of work entirely. **Even the most basic allow/block equivalent (`NEFilterDataProvider`) requires shipping our own extension code**, same as the redirect-capable `NETransparentProxyProvider` does — both run in **user space**, just two different extension types/bundles.

Blast radius — structural, not incidental: a bug in the Windows callout can bluescreen the whole machine (kernel-mode). A bug in the macOS extension kills one user-space process; the OS keeps running. This difference exists because of how each OS lets third parties intervene in networking, not because of how carefully either would be written.

Total work, compared honestly: - *Windows:* `block/allow` — already shipped, zero new work. Redirect — one new kernel driver plus signing (EV certificate + attestation signing, or WHQL), but it **reuses** the existing process-attribution and DNS -correlation infrastructure. One investment covers both tiers. - *macOS:* `block/allow` — daemon-side plumbing exists (`net_egress/macos.rs`) but the actual `NEFilterDataProvider` extension doesn't. Redirect needs a **separate, third** extension bundle (distinct from both the existing Endpoint Security extension and the not-yet-built filter one). Pursuing both tiers on macOS means shipping **two** extensions eventually, not one — a larger total lift than Windows's single-driver path, even though each individual macOS piece is lower-risk (user-space, no bluescreen exposure).

Options considered, per platform: - *Windows:* no real alternative surfaced — WFP is the OS-sanctioned interception point, full stop. The only design question was which layer (`ALE_AUTH_CONNECT_V4` , already used, vs. `ALE_CONNECT_REDIRECT_V4` , new). - *macOS:* three Network Extension provider types were weighed — `NEFilterDataProvider` (insufficient alone, no redirect), `NEDNSProxyProvider` (can fake DNS dynamically, but system-wide in scope, not app-scoped — rejected for that reason), and `NETransparentProxyProvider` (chosen — the only one with both redirect and `NEAppRule` app-scoping). Non-extension alternatives were also tried and explicitly ruled out this session: a static `/etc/hosts` override (system-wide, manual cleanup, no app-scoping), `DYLD_INSERT_LIBRARIES` dylib injection (blocked by Hardened Runtime on a properly signed third-party app), `Info.plist / LSEnvironment` edits and binary-swap wrapper tricks (both break code signing and get wiped by the app's own auto-update), and a user-launched wrapper script (works, but needs a human to cooperate every time — not true enforcement).